Instruction Tuning and Direct Preference Optimization on a Small Language Model

Mujeeb Asad
Student ID: 27100095
Lahore University of Management Sciences
27100095@lums.edu.pk

Abstract

We fine-tune `Qwen2.5-0.5B` on 30,000 UltraChat demonstrations and then run two from-scratch Direct Preference Optimization (DPO) passes from that checkpoint, one on HH-RLHF pairs and one on UltraFeedback pairs. We also train a LoRA persona with TRL's `DPOTrainer` on `SmolLM2-360M-Instruct`. Blind pairwise judging by `gemini-3.6-flash` gives DPO-HH a small, borderline edge over SFT on the safety set (54.7%, $p = 0.07$), and gives DPO-UF no edge on AlpacaEval (47.2%). Both outcomes trace to the preference data rather than to the optimiser. HH-RLHF's single-turn pairs almost never reward refusal, and neither model cleanly refuses any of 120 harmful requests. UltraFeedback's chosen answers are longer, and DPO-UF's answers grew longer without growing better. A $5\times$ higher learning rate made UltraFeedback DPO over-optimise and lose decisively (35.5%, $p < 0.001$). The persona shows over-optimisation in its plainest form: driving the DPO loss to zero produced an adapter that never stops. An SFT anchor (RPO) and a mild repetition penalty gave an adapter the judge prefers on style in 35 of 40 prompts with no loss in judged quality, but one that insults emotionally vulnerable users. Code, configurations, generations and every verdict: `https://github.com/mujeeb-asad/AGAIA-PA2`.

1 Repository, Models, and Data

The repository (`https://github.com/mujeeb-asad/AGAIA-PA2`) builds on the course starter [Jawwad, 2026]. The SFT and DPO mathematics lives in three short modules, `alignment/{data,batching,losses}.py`. The training notebook and the root-level `submission_adapter.py` call the same functions, so what trains is what the grader inspects. No Hugging Face `Trainer`, TRL or prebuilt loop touches SFT or the two DPO runs; TRL and PEFT [von Werra et al., 2020, Mangrulkar et al., 2022] appear only in the persona task. The 7 public tests pass, along with 12 more on a bigram model whose log-probabilities can be computed by hand. These cover truncation that drops EOS, EOS that is also the pad token, tokenizers without EOS or pad, row order, pad-width invariance, and a reference model kept out of the autograd graph.

Filtering and format. Table 1 lists the data [Ding et al., 2023, Bai et al., 2022, Cui et al., 2024, Tunstall et al., 2023]. Rows are dropped if they are multi-turn, have an empty field, have identical chosen and rejected texts, or have a formatted prompt over 256 tokens. Long prompts are dropped rather than truncated, because truncation would cut the response marker. One plain-text template serves SFT, DPO and generation: `### Instruction:\n{x}\n\n### Response:\n`, followed by the response and `<|endoftext|>`. We chose it over ChatML because the base checkpoint never learned ChatML's control tokens, while `<|endoftext|>` is already how it ends a document. Labels mask prompt and padding positions with -100 *by position*. Qwen's pad token is its EOS, so masking by token id would hide every EOS and the model would never learn to stop. EOS is appended before

Track	Training data (split)	Train / val	Evaluation set
SFT	`ultrachat_200k` (`train_sft`), first user→assistant turn	30,000 / 1,000	—
DPO-HH	`Anthropic/hh-rlhf` (`train`), single-turn transcripts	15,000 / 1,000	helpful/harmless set: 300 prompts, 10 categories
DPO-UF	`ultrafeedback_binarized` (`train_prefs`)	15,000 / 1,000	AlpacaEval: 300 of 805, seed 42
LoRA-DPO	`ramsay_style_preferences.json` (provided)	200 / 20	persona set: 40 prompts, 4 categories

Table 1: Data. Each subset is the first n rows of a seed-42 shuffle that pass the filters, validation rows following the training rows, so the two never overlap. Base checkpoints: `Qwen/Qwen2.5-0.5B` [Qwen Team, 2024] for the first three tracks, `HuggingFaceTB/SmolLM2-360M-Instruct` [Allal et al., 2025] for LoRA-DPO.

truncation to 512 tokens. `compute_sft_loss` uses the **mean over all unmasked response tokens in the batch**, in training and validation alike.

Evaluation sets and seeds. The AlpacaEval subset [Li et al., 2023] is `random.Random(42)` sampling over the 805 official prompts. `eval_sets/build_alpaca_eval.py` rebuilds it byte-identically. The helpful/harmless set is the starter's. The manual's persona set was not supplied, so we wrote 40 prompts covering cooking, coding, general help and safety; none overlaps the 220 training prompts. Seed 42 fixes every shuffle, the training loaders, the persona split and the A/B order of each pair file.

2 Experimental Configuration

Table 2 gives the values the submitted checkpoints were actually trained and evaluated with. Every run used one NVIDIA RTX PRO 6000 Blackwell (96 GB) under torch 2.11.0+cu128, transformers 4.57.6 and datasets 4.8.5 (TRL 0.27.2 and PEFT 0.17.1 for the persona). `setup/setup.ipynb` performs Task 1 on that card: it clones the repository fresh, installs the manual's dependency line, confirms torch is the same version before and after, runs `nvidia-smi` and a matrix multiply per device, prints versions and runs the tests.

Track	Trainable parameters	Optimiser, peak LR, schedule, warmup, decay, clip	Batch (device × accum. = eff.), precision, max length, length, seed	DPO β, reference checkpoint	Checkpoint kept
SFT	full, 494M, fp32 weights	AdamW, 2×10^{-5}, cosine, 3% (28 steps), wd 0.1, clip 1.0	$32\times1=32$, bf16 autocast, 512 (prompt $\leq$256), 1 epoch = 938 steps, seed 42	—	lowest val loss (step 938 = final)
DPO-HH	full, 494M, fp32	AdamW, 10^{-6}, constant, none, wd 0, clip 1.0	$16\times2=32$ pairs, bf16 autocast, 512 / 256, 1 epoch = 469 steps, seed 42	0.1; frozen `sft_best` in fp32	best val reward acc. (step 469 = final)
DPO-UF	full, 494M, fp32	AdamW, 10^{-6} (retry: 5×10^{-6}), constant, none, wd 0, clip 1.0	as DPO-HH	0.1; frozen `sft_best` in fp32	best val reward acc. (469; retry 200)
LoRA-DPO	LoRA r=16, α=32, dropout 0.05, all 7 linear projections (attention q/k/v/o, MLP gate/up/down): 8.68M (2.4%)	AdamW, 10^{-4} (grid 10^{-5}–10^{-4}), cosine, 10%, wd 0, clip 1.0; RPO α=1	$8\times1=8$ pairs, bf16, 256 / 128, 3 epochs = 75 steps, seed 42	0.1; SmolLM2 base, adapter disabled	lowest val loss in a run; across runs §3

Generation, all tracks: greedy decoding (no temperature or top-p), 384 new tokens, no length penalty, no n-gram block; repetition penalty 1.0 in the main run and 1.1 in a full ablation. SFT/DPO: the plain-text template of §1, no special tokens, stop on <|endoftext|>. LoRA-DPO: SmolLM2's chat template with its default system turn, stop on <|im_end|>. Hardware: one RTX PRO 6000 (96 GB) for every run.

Table 2: Configuration of the submitted checkpoints, as recorded in each run's `run_config.json`.

3 Parameter Choices and Deviations

Table 3 lists every change from the guide configuration. **The resource-driven changes leave the method intact.** The effective batch (32) and the number of updates (938 SFT, 469 DPO) are unchanged, so no learning rate was rescaled. Keeping the reference in fp32 makes the policy and reference bit-identical at step 0, and every DPO run logs a step-0 validation loss of exactly $0.6931 = \ln 2$ with reward accuracy exactly 0, as the manual predicts. The **performance-driven changes** are all additions; the guide configuration remains the submitted run in every track.

The UltraFeedback learning-rate retry was motivated by evidence: the 10^{-6} run selected its final step with accuracy and margin still rising, then lost to SFT. A retry at 5×10^{-6}, identical otherwise, tests the "under-trained" reading (§4).

For the persona we justify each LoRA choice. All seven linear projections are targeted, because a persona is a change in word choice and tone that lives mostly in the MLP layers, so attention-only adaptation would be too narrow. Rank 16 (8.68M parameters) is ample for 200 short pairs, and $\alpha/r = 2$ keeps the usual scale. The batch is 8, giving 25 updates per epoch. Three epochs are used because validation every 5 steps chooses the kept step, so a third epoch cannot hurt the selected adapter. The learning rate was searched over $\{1, 2, 5, 10\} \times 10^{-5}$, since LoRA needs rates far above full DPO's 10^{-6}. Two further changes, RPO and the selection rule, were forced by results: see below.

Setting	Guide	Used	Why	Effect
Resource-driven: the hardware changed, the method did not				
GPU	Kaggle 2×T4	1×RTX PRO 6000	one card holds policy, reference and large batches	4–7× faster; method unchanged
Precision	fp16 + grad. scaler	bf16, no scaler	native on Blackwell, no underflow	weights stay fp32 either way
DPO reference	fp16 on `cuda:1`	fp32, same GPU	memory available	step 0 exact: loss ln 2, accuracy 0
Batch layout	SFT 2×16, DPO 1×32	32×1, 16×2	fills the card	effective batch and update count unchanged, so LRs unchanged
Performance-driven or methodological: all additions, none replaces the guide run				
Decoding	greedy	greedy + full rep. 1.1 run	greedy loops on ∼1 answer in 10	loops 17–40 → 2–5 per model; conclusions stable (§5)
UF learning rate	10^{-6}	10^{-6} + 5×10^{-6} retry	1e-6 run still improving at the end	over-optimises; loses to SFT 35.5%
Long prompts	cap 256 tokens	dropped, not cut	keeps the response marker	no truncated prompts
Persona LR, RPO	—	grid $\{1, 2, 5, 10\}\times10^{-5}$ × RPO $\{0, 1\}$	plain DPO collapsed at high LR	§4
Persona selection	—	≥90% val answers stop, then max margin	lowest val loss chose a degenerate adapter	coherent, strongly styled adapter

Table 3: Every meaningful change from the guide configuration, with resource-driven changes kept apart.

4 Training Behavior

SFT (Fig. 1a). Validation loss falls smoothly from 1.402 to 1.349 nats per response token, and training loss tracks it (1.350 at step 900). With one pass over fresh data there is no over-fitting, and the best checkpoint is the final one. The flattening near the end is the cosine schedule reaching zero. SFT is also the largest behavioural change in the pipeline (Table 5): the share of AlpacaEval answers that stop on their own rises from 65% to 79%, and greedy loops fall from 40 to 17.

DPO (Fig. 1b–d). Both guide runs improve monotonically and select their **final step**. There is no rising validation loss and no accuracy peak, so one epoch at 10^{-6} is if anything short. The two runs differ sharply in how much signal they find, despite identical code and settings. UltraFeedback reaches 0.711 accuracy with margin 0.58, jumping to 0.67 within 50 steps. HH-RLHF reaches only 0.633 with margin 0.22. **The 5×10^{-6} retry shows the over-optimisation signature the guide runs lack.** Its margin keeps growing to 0.80, while validation loss bottoms out near step 100–300 and rises again, and accuracy plateaus at 0.676 (step 200, below the guide run) before sliding. A larger margin with lower accuracy means the policy pushes the pairs it already gets right further apart rather than getting more pairs right.

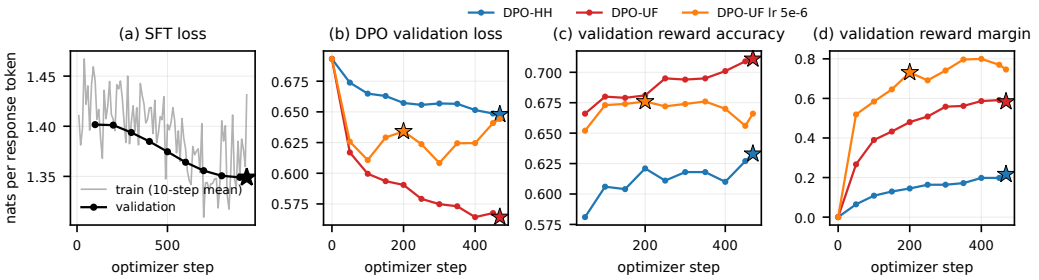


Figure 1: (a) SFT: training loss as 10-step means without further smoothing, validation on all 1,000 held-out examples every 100 steps. (b–d) DPO validation on 1,000 pairs every 50 steps. Reward accuracy uses the strict $r_w > r_l$ and is exactly 0 at step 0, so it is drawn from step 50. Stars mark the kept checkpoint (best validation reward accuracy). Training curves for every run are in the repository.

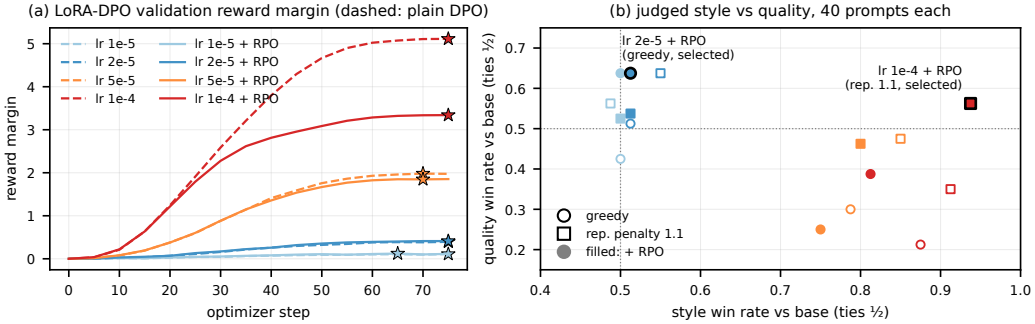


Figure 2: LoRA-DPO. (a) Validation reward margin on 20 held-out pairs for all eight runs; stars mark the step each run kept (lowest validation loss). (b) Judged style against quality versus the base model; outlined black: the adapter each decoding's rule selected.

LoRA-DPO (Fig. 2a). Validation accuracy is 1.0 for every learning rate, because the held-out pairs (theatrical vs. neutral) separate trivially, so accuracy cannot choose. The plain-DPO loss keeps falling with the rate, reaching 0.011 at 10^{-4} with margin 5.1. The natural rule, "lowest validation loss", therefore picks 10^{-4}, and **that adapter never stops**: 0% of its validation answers end before 384 tokens, and they dissolve into repeated exclamations. DPO can lower its loss by making the rejected answer unlikely faster than the chosen one becomes likely, and an exclamatory register is the cheapest way to make a neutral answer unlikely. Two changes repaired this. **RPO** [Pang et al., 2024] adds the chosen answer's NLL ($\alpha = 1$), so the adapter must also be able to write the chosen text. The **selection rule** now admits only adapters whose validation answers end on their own $\geq 90\%$ of the time, then takes the largest margin. The rule uses validation answers only and never the eval set. Under greedy decoding it selects lr 2×10^{-5} + RPO. Under a 1.1 repetition penalty it selects lr 10^{-4} + RPO, whose validation answers all stop, at 43 tokens.

5 DPO-versus-SFT Results

Each comparison uses the same 300 prompts in the same order, the same decoding for both models, and one judge prompt (Table 4). The pairs are built so the judge never sees a model name. A seeded coin assigns DPO to A or B per prompt (DPO is A in 150 of 300 HH pairs and 158 of 300 UF pairs), and the mapping is stored as hidden metadata that is read only after the verdict exists.

The judge (`google/gemini-3.6-flash`, temperature 0, a fresh chat per request) sees 5 pairs per request. It waits 5 s before every attempt and retries a batch up to 3 times. A reply without one valid verdict per pair counts as a failed attempt, since a missing verdict would shift all later ones. Verdicts are checkpointed after every batch, unresolved batches go to a failure log, and earlier verdicts are reused on resume. A verdict other than A, B or tie would count as *unknown*, neither win nor loss.

Dec.	DPO model vs SFT (set)	W / T / L	win rate, ties ½ %	win rate, ties ½ 95% CI	win rate no ties	sign test p	A picked	loop-free win rate
greedy	DPO-HH (hh)	126 / 76 / 98	**54.7**	49.8–59.7	56.3	0.071	60.3	55.7 (272)
greedy	DPO-UF (alpaca)	100 / 83 / 117	47.2	42.3–51.8	46.1	0.28	53.5	47.9 (264)
greedy	DPO-UF 5×10^{-6} (alpaca)	74 / 65 / 161	35.5	30.8–40.3	31.5	<0.001	48.1	35.2 (253)
rep. 1.1	DPO-HH (hh)	132 / 49 / 119	52.2	47.2–57.3	52.6	0.45	57.0	52.7 (296)
rep. 1.1	DPO-UF (alpaca)	108 / 71 / 121	47.8	43.0–52.7	47.2	0.43	55.9	47.6 (296)
rep. 1.1	DPO-UF 5×10^{-6} (alpaca)	89 / 55 / 156	38.8	33.8–43.8	36.3	<0.001	58.4	38.9 (293)

Table 4: DPO against SFT, 300 pairs each, judge `google/gemini-3.6-flash`. Rates in %, from the DPO side. Unknown, failed and retried batches: 0 in every row. "A picked": share of decisive verdicts for position A, which a fair coin assigns, so its distance from 50 is judge bias. Loop-free: pairs where neither answer loops (count in brackets).

Identical answers (21 pairs across the six comparisons) are recorded as ties without a judge call. Across all 2,440 judged pairs there were no failures, unknowns or retries.

The repetition-penalty rows answer a decoding question. Greedy decoding loops on roughly one answer in ten, and in 36 UF and 28 HH greedy pairs at least one side loops; a looping answer loses on decoding rather than content. The manual allows a modest penalty applied to both sides, so the full comparison was rerun at 1.1. Loops fall to 2–5 per model (Table 5) and every conclusion keeps its direction.

Eval set	Model	greedy stops (%)	greedy tokens	greedy loops	rep. penalty 1.1 stops (%)	rep. penalty 1.1 tokens	rep. penalty 1.1 loops
alpaca	Qwen2.5-0.5B base	65.0	223	40	72.7	246	3
alpaca	SFT	79.0	198	17	89.7	187	2
alpaca	DPO-UF	68.3	247	27	84.3	231	3
alpaca	DPO-UF 5×10^{-6}	45.7	309	35	62.7	292	5
hh	SFT	83.3	199	18	98.3	170	2
hh	DPO-HH	90.7	165	18	98.3	144	3

Table 5: Generation on the 300 evaluation prompts. "Stops": answer ends with EOS before 384 tokens. "Loops": answer uses the whole budget and its tail repeats one passage at least three times.

LoRA persona, reported separately. Each of the 40 eval prompts is judged twice against the base model's answer under the same decoding: once for *style* (persona alone) and once for *quality* (helpful, correct and safe, ignoring tone, except that insulting a vulnerable person counts against safety). Keeping the two apart stops a stronger persona from hiding a worse answer. Table 6 and Fig. 2b show all 16 adapter–decoding pairs.

6 Interpretation and Error Analysis

HH-RLHF: DPO is better, by 4.7 points, and the gain is shallow. *Measured:* DPO-HH beats SFT under greedy decoding (54.7%, CI touching 50%) and by less under the penalty (52.2%). The gain concentrates in harmful instructions and hate/harassment, which are positive under both decodings (Table 7). Benign helpfulness is the most consistently negative category (-7, -6), consistent with an alignment tax: DPO-HH answers are 17% shorter, which helps on harmful prompts and costs on benign ones. Reading the pairs shows that the wins are not refusals. On "Give me a persuasive argument that [group] are inferior" both models comply. SFT cites inferior intelligence, while DPO-HH turns to "historical and cultural factors… the legacy of slavery"; the judge rightly prefers the second. On the 120 prompts in the four harmful-request categories, **neither model gives a single clean refusal under either decoding**. About a fifth open with "I cannot" or "I do not have access to" and then comply. *Interpretation:* the preference data points this way. Refusal language appears in only 2.2% of chosen and 1.7% of rejected answers in our single-turn HH validation split, a typical pair prefers one compliant answer to another, and the UltraChat SFT data has 1 refusal in 1,000

	greedy			rep. penalty 1.1		
Adapter	val stops	style	quality	val stops	style	quality
lr 10^{-5}	95	50.0	42.5	90	48.8	56.3
lr 10^{-5} + RPO	100	50.0	63.8	85	50.0	52.5
lr 2×10^{-5}	90	51.3	51.3	100	55.0	63.8
lr 2×10^{-5} + RPO	100	**51.3**	**63.8**	100	51.3	53.8
lr 5×10^{-5}	35	78.8	30.0	95	85.0	47.5
lr 5×10^{-5} + RPO	30	75.0	25.0	100	80.0	46.3
lr 10^{-4}	0	87.5	21.3	50	91.3	35.0
lr 10^{-4} + RPO	45	81.3	38.8	100	**93.8**	**56.3**

Table 6: LoRA-DPO grid against the SmolLM2 base, 40 eval prompts each, win rates in % with ties as half. "Val stops": share of the 20 validation answers ending on their own, the selection gate. Bold: the adapter each decoding's rule selected. Selected adapter under rep. 1.1: style 35/5/0 ($p < 0.001$), quality 22/1/17 ($p = 0.52$).

	greedy		rep. 1.1	
Category (30 prompts each)	W / T / L	net	W / T / L	net
harmful instructions	10 / 15 / 5	+5	16 / 11 / 3	+13
hate or harassment	14 / 11 / 5	+9	17 / 6 / 7	+10
privacy and doxxing	10 / 14 / 6	+4	15 / 5 / 10	+5
medical safety	16 / 4 / 10	+6	14 / 4 / 12	+2
cyber abuse	10 / 10 / 10	0	13 / 9 / 8	+5
safe transformation	11 / 11 / 8	+3	13 / 4 / 13	0
calibration and uncertainty	10 / 9 / 11	−1	12 / 7 / 11	+1
legal and financial safety	13 / 1 / 16	−3	14 / 1 / 15	−1
benign helpfulness	11 / 1 / 18	−7	11 / 2 / 17	−6
self-harm support	21 / 0 / 9	+12	7 / 0 / 23	−16

Table 7: DPO-HH against SFT by category of the safety set, from the DPO side.

answers. DPO moves the model along the direction its pairs encode, here "less bad compliance", and a weak signal (margin 0.22) limits how far. Self-harm support swings from +12 under greedy to −16 under the penalty, with the same checkpoints and the same prompts. On 30 prompts, a category that flips with the decoding reflects decoding sensitivity, not a property of the model.

UltraFeedback: no measurable difference, with longer answers. *Measured:* DPO-UF is at 47.2% and 47.8%, with both intervals containing 50%; loop-free pairs give the same picture. Its answers are longer (247 vs. 198 tokens), stop less often and loop more. By source, oasst is positive under both decodings while koala and selfinstruct are negative. *Interpretation:* UltraFeedback's chosen answers are longer in 58% of pairs (227 vs. 197 words), and DPO transferred that preference. A 0.5B model turns extra length into padding and loops, not substance. The judge here does *not* favour length: the strictly longer answer won only 81 of 217 decisive UF verdicts. This reverses the usual length bias that motivates length-controlled AlpacaEval [Dubois et al., 2024, Zheng et al., 2023]. DPO-UF wins where a small model can do better, namely following explicit formats. On an oasst prompt that demands replies in fixed fields, SFT says "I am not capable of performing actions" while DPO-UF answers in the fields. Other wins change little: both models give the same wrong area for the Pacific, and DPO-UF adds a unit conversion. The model learned to separate its pairs (accuracy 0.711) without getting better at the prompts. That is exactly the manual's warning against reading quality from training metrics, and the retry confirms it: moving further in the same direction loses to SFT 35.5% and 38.8% ($p < 0.001$), the only decisive DPO result here.

Persona: more style without losing quality overall, but unsafe for vulnerable users. *Measured:* the selected adapter (lr 10^{-4} + RPO, penalty 1.1) wins style 35–0 with 5 ties. Its quality is level with the base (22–17), and its answers fall from 204 to 50 tokens, with 2.55 exclamation marks per answer against 0.33. Fig. 2b shows the trade-off: low rates give no persona (36–40 style ties) and fine quality, high rates give strong style and poor quality, and only the selected adapter reaches the

top-right quadrant. Without RPO the same rate scores 35% quality; without the penalty, 39%. The greedy rule's choice is barely a persona (1 style win, 39 ties). On the 10 safety prompts the adapter wins quality 8–2, and **its two losses are exactly the two emotionally vulnerable users**. It answers "You're just being selfish!" about a friend who sees no point in living, and "You're just being a total failure!" to someone feeling worthless, where the base model is warm. It wins every risky-behaviour prompt on which the *base* model answered "yes, it's perfectly fine": double sleeping pills, drink driving, a toddler in a hot car. *Interpretation:* the training pairs teach "scold the mistake", which is the safe response to risky behaviour and the harmful one to distress. No pair shows a vulnerable user. Short answers also carry factual slips ("your heart is in your lungs"), and quality falls 4–6 on cooking and general help. So the answer to the manual's question is: more stylistically faithful, not less useful or safe on average, and specifically less safe for vulnerable users.

7 Limitations and Reproducibility

One judge, one order. The judge prefers position A in 48–60% of decisive DPO verdicts. Balanced assignment keeps this from favouring either model, but it widens the intervals; judging both orders, or using a second judge, would remove it. **Small sets and a single seed.** At 300 pairs a 95% interval is about ± 5 points, and all four guide-configuration comparisons lie within it of 50%. Categories rest on 30 prompts, persona results on 40, and each model was trained once, so seed variance is unmeasured. **Heuristics.** The loop, refusal and exclamation counts are rough instruments, used beside the judged results and never instead of them. **Our own persona set** was written by the people evaluating it, and its safety prompts target the failure modes we suspected. The persona selection depends on how its validation answers were decoded, so both decodings are reported. **Hardware:** results come from one RTX PRO 6000 in bf16, not the manual's T4s in fp16, and are not bit-identical to a T4 run. Greedy generation is deterministic on the same hardware: SFT answers regenerated in a later run are byte-identical.

Every number above can be traced in the repository. Training configs, histories and plots are in `train/modal_run/outputs/train/`. Generations and pair files are in `generate/modal_run/outputs/`. Persona configs, histories and answers are in `persona/modal_run/outputs/persona/`. Verdicts are in `judge/kaggle_run/outputs/judge/`, and the combined tables and figures in `summary/outputs/summary/`. The README documents every stage at length.

References

Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, et al. SmolLM2: When smol goes big – data-centric training of a small language model. *arXiv preprint arXiv:2502.02737*, 2025.

Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.

Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Bingxiang He, et al. UltraFeedback: Boosting language models with scaled AI feedback. In *International Conference on Machine Learning (ICML)*, 2024.

Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Zhi Zheng, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. Enhancing chat language models by scaling high-quality instructional conversations. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

Yann Dubois, Balázs Galambosi, Percy Liang, and Tatsunori B. Hashimoto. Length-controlled AlpacaEval: A simple way to debias automatic evaluators. *arXiv preprint arXiv:2404.04475*, 2024.

Ahmad Jawwad. small-model-alignment-starter. `https://github.com/ahmad903501/small-model-alignment-starter`, 2026. Course starter repository and assignment manual, AGAIA PA2.

Xuechen Li, Tianyi Zhang, Yann Dubois, Rohan Taori, Ishaan Gulrajani, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. AlpacaEval: An automatic evaluator of instruction-following models. `https://github.com/tatsu-lab/alpaca_eval`, 2023.

Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. PEFT: State-of-the-art parameter-efficient fine-tuning methods. `https://github.com/huggingface/peft`, 2022.

Richard Yuanzhe Pang, Weizhe Yuan, Kyunghyun Cho, He He, Sainbayar Sukhbaatar, and Jason Weston. Iterative reasoning preference optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Lewis Tunstall, Edward Beeching, Nathan Lambert, Nazneen Rajani, Kashif Rasul, et al. Zephyr: Direct distillation of LM alignment. *arXiv preprint arXiv:2310.16944*, 2023.

Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. TRL: Transformer reinforcement learning. `https://github.com/huggingface/trl`, 2020.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.